uniqueConstraintDuplicateCrewUpdate — AI-Assisted Enhancement Report

File: `uniqueConstraintDuplicateCrewUpdate.ts`
Purpose: Clear duplicate `passport_no`, `seaman_book_no`, and `payroll_no` values on crew records before unique constraints are applied (runs at server startup).

---

1. Original Code

Export name: `uniqueDuplicateCrewUpdate` (~104 lines, single function, `any` types)

```
import logger from '../config/loggerConfig';
import { col, fn, literal } from 'sequelize';
import CrewDetails from '../models/crewDetails';

export const uniqueDuplicateCrewUpdate = async (): Promise<string> => {
  try {
    logger.info(`Crew Unique Constraint Duplicate Update Start`);

    const passportDuplicateQuery = await CrewDetails.findAll({
      attributes: ['passport_no',
        [fn('COUNT', col('passport_no')), 'count'],
        [fn('array_agg', col('id')), 'ids'],
      ],
      group: ['passport_no'],
      having: literal('COUNT("passport_no") > 1'),
    })

    const seamanDuplicateQuery = CrewDetails.findAll({
      attributes: ['seaman_book_no',
        [fn('COUNT', col('seaman_book_no')), 'count'],
        [fn('array_agg', col('id')), 'ids'],
      ],
      group: ['seaman_book_no'],
      having: literal('COUNT("seaman_book_no") > 1'),
    })

    const payrollDuplicateQuery = CrewDetails.findAll({
      attributes: ['payroll_no',
        [fn('COUNT', col('payroll_no')), 'count'],
        [fn('array_agg', col('id')), 'ids'],
      ],
      group: ['payroll_no'],
      having: literal('COUNT("payroll_no") > 1'),
    })

    const [passportDuplicateData,
      seamanDuplicateData,
      payrollDuplicateData] =
      await Promise.all([passportDuplicateQuery,
        seamanDuplicateQuery,
        payrollDuplicateQuery]);

    if(!(passportDuplicateData?.length &&
      seamanDuplicateData?.length &&
```

```
      payrollDuplicateData?.length)) {
      logger.info(`No Crew Duplicate To Update -
        Duplicate length below
        Passport ${passportDuplicateData?.length},
        Seaman ${seamanDuplicateData?.length},
        Payroll ${payrollDuplicateData?.length}`);
      return "";
    }

    let duplicateUpdate: any = {};

    if (passportDuplicateData?.length) {
      passportDuplicateData.forEach((data: any) => {
        (data?.dataValues?.ids || []).forEach((id: any) => {
          duplicateUpdate[id] = {
            passport_no: null
          }
        })
      })
    }

    if (seamanDuplicateData?.length) {
      seamanDuplicateData.forEach((data: any) => {
        (data?.dataValues?.ids || []).forEach((id: any) => {
          if (duplicateUpdate[id])
            duplicateUpdate[id].seaman_book_no = null;
          else duplicateUpdate[id] = {
            seaman_book_no: null
          }
        })
      })
    }

    if (payrollDuplicateData?.length) {
      payrollDuplicateData.forEach((data: any) => {
        (data?.dataValues?.ids || []).forEach((id: any) => {
          if (duplicateUpdate[id])
            duplicateUpdate[id].payroll_no = null;
          else duplicateUpdate[id] = {
            payroll_no: null
          }
        })
      })
    }

    for (const crewId in duplicateUpdate) {
      await CrewDetails.update(
        duplicateUpdate[crewId], {
        where: {
          id: crewId
        },
      })
      logger.info(`Crew Duplicate Update successfully - id ${crewId}`);
    }

    return 'Successfully Updated'
  } catch (e) {
    throw e;
  }
}
```

Known issues in original code

Issue	Impact
Early exit uses **AND** (all three fields must have duplicates)	Skips cleanup when only one field has duplicates
No filter for null/empty or deleted crew	May null blank or deleted rows incorrectly
No database **transaction**	Partial updates on failure
`any` types, useless `try/catch`	Weak typing and noise
Sequential updates, no `raw` aggregates	Slower; fragile `ids` parsing
Misnamed export vs file name	Import confusion

---

2. AI Prompts Used (Cursor / Composer)

Prompts were applied iteratively in one session:

1. *Review the file uniqueConstraintDuplicateCrewUpdate.ts, and show the edge cases … without changing code yet.*
2. *Apply the fixes for the edge cases with minimal difference*
3. *Remove the useless try/catch that only rethrows … Do not change behavior otherwise.*
4. *Rename the export so it matches the file name (uniqueConstraintDuplicateCrewUpdate) and update the imports*
5. *Replace 'any' … with proper TypeScript types. Match existing patterns in crewDetails model*
6. *Refactor … with explicit error handling per function*
7. *Wrap all crew updates in a DB transaction so partial failure doesn't leave some crews updated*
8. *Add a very short JSDoc on the exported function …*
9. *Improve the performance optimization without any logic changes*
10. *Review for regressions and missing edge cases*
11. *Apply the fixes for the above missing cases*
12. *Please refactor and fix the TSLint errors of this file alone*

---

3. Enhanced Code

Export name: `uniqueConstraintDuplicateCrewUpdate` (~321 lines, modular helpers, typed, transactional)

Path: `C:\Users\daisy.a\Desktop\uniqueConstraintDuplicateCrewUpdate.ts`

```
import logger from '../config/loggerConfig';
import {
  col,
  fn,
  literal,
  Op,
  Transaction,
```

```
  WhereOptions,
} from 'sequelize';
import sequelize from '../data/db';
import CrewDetails, { CrewListAttributes } from '../models/crewDetails';

const UPDATE_BATCH_SIZE = 50;

const UUID_REGEX =
  /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i;

type DuplicateUniqueField = keyof Pick<
  CrewListAttributes,
  'passport_no' | 'seaman_book_no' | 'payroll_no'
>;

const DUPLICATE_FIELDS: readonly DuplicateUniqueField[] = [
  'passport_no',
  'seaman_book_no',
  'payroll_no',
] as const;

type DuplicateAggregateRow = {
  count?: string | number;
  ids?: string[] | string | number | number[];
  dataValues?: {
    ids?: string[] | string | number | number[];
    count?: string | number;
  };
} & Partial<Pick<CrewListAttributes, DuplicateUniqueField>>;

type CrewDuplicateNullUpdate = Partial<{
  passport_no: null;
  seaman_book_no: null;
  payroll_no: null;
}>;

type DuplicateUpdateMap = Record<CrewListAttributes['id'], CrewDuplicateNullUpdate>;

type DuplicateGroupsResult = Record<
  DuplicateUniqueField,
  DuplicateAggregateRow[]
>;

const getErrorMessage = (error: unknown): string =>
  error instanceof Error ? error.message : String(error);

const logAndRethrow = (context: string, error: unknown): never => {
  logger.error(`${context}: ${getErrorMessage(error)}`);
  throw error;
};

const runAsync = async <T>(context: string, fn: () => Promise<T>): Promise<T> => {
  try {
    return await fn();
  } catch (error: unknown) {
    return logAndRethrow(context, error);
  }
};

const runSync = <T>(context: string, fn: () => T): T => {
  try {
    return fn();
  } catch (error: unknown) {
    return logAndRethrow(context, error);
  }
};

const normalizedFieldExpr = (field: DuplicateUniqueField) =>
```

```
  fn('LOWER', fn('TRIM', col(field)));

const activeCrewWhere = (): WhereOptions<CrewListAttributes> =>
  ({
    [Op.or]: [
      { is_deleted: false },
      { is_deleted: { [Op.is]: null } },
    ],
  }) as WhereOptions<CrewListAttributes>;

const nonEmptyFieldWhere = (field: DuplicateUniqueField): WhereOptions<CrewListAttributes> => ({
  [Op.and]: [
    activeCrewWhere(),
    { [field]: { [Op.ne]: null } },
    { [field]: { [Op.ne]: '' } },
    sequelize.where(normalizedFieldExpr(field), { [Op.ne]: '' }),
  ],
});

const parsePostgresArrayString = (value: string): string[] => {
  const trimmed = value.trim();
  if (!trimmed.startsWith('{') || !trimmed.endsWith('}')) {
    return [];
  }
  const inner = trimmed.slice(1, -1).trim();
  if (!inner) {
    return [];
  }
  return inner
    .split(',')
    .map((part) => part.trim().replace(/^"(.*)"$/, '$1'))
    .filter((part) => part.length > 0);
};

const getRawIdsFromRow = (row: DuplicateAggregateRow): unknown =>
  row.ids !== undefined ? row.ids : row.dataValues?.ids;

const parseAggregatedIds = (row: DuplicateAggregateRow): CrewListAttributes['id'][] =>
  runSync('Failed to parse aggregated crew ids', () => {
    const raw = getRawIdsFromRow(row);
    let parsed: string[] = [];

    if (Array.isArray(raw)) {
      parsed = raw.map(String);
    } else if (typeof raw === 'string') {
      const fromPgArray = parsePostgresArrayString(raw);
      parsed = fromPgArray.length > 0 ? fromPgArray : [raw];
    } else if (raw != null) {
      parsed = [String(raw)];
    }

    return parsed
      .map((id) => id.trim())
      .filter((id) => UUID_REGEX.test(id));
  });

const getDuplicateGroupLabel = (
  row: DuplicateAggregateRow,
  field: DuplicateUniqueField,
): string => String(row[field] ?? 'unknown');

const setFieldNullOnCrew = (
  duplicateUpdate: DuplicateUpdateMap,
  crewId: CrewListAttributes['id'],
  field: DuplicateUniqueField,
): void => {
  const existing = duplicateUpdate[crewId];
  duplicateUpdate[crewId] = existing
```

```
    ? { ...existing, [field]: null }
    : { [field]: null };
};

const findDuplicateGroups = (
  field: DuplicateUniqueField,
): Promise<DuplicateAggregateRow[]> =>
  runAsync(`Failed to find duplicate crew groups for ${field}`, async () => {
    const normalized = normalizedFieldExpr(field);
    const rows = await CrewDetails.findAll({
      attributes: [
        [normalized, field],
        [fn('COUNT', col('id')), 'count'],
        [literal('array_agg("id" ORDER BY "id" ASC)'), 'ids'],
      ],
      where: nonEmptyFieldWhere(field),
      group: [normalized],
      having: literal('COUNT("id") > 1'),
      raw: true,
      subQuery: false,
    });
    return rows as DuplicateAggregateRow[];
  });

const findAllDuplicateGroups = (): Promise<DuplicateGroupsResult> =>
  runAsync('Failed to load duplicate crew groups', async () => {
    const [passport, seaman, payroll] = await Promise.all(
      DUPLICATE_FIELDS.map((field) => findDuplicateGroups(field)),
    );
    return {
      passport_no: passport,
      seaman_book_no: seaman,
      payroll_no: payroll,
    };
  });

const hasDuplicateGroups = (groups: DuplicateGroupsResult): boolean =>
  DUPLICATE_FIELDS.some((field) => groups[field].length > 0);

const markDuplicateIdsForNulling = (
  rows: DuplicateAggregateRow[],
  field: DuplicateUniqueField,
  duplicateUpdate: DuplicateUpdateMap,
): void =>
  runSync(`Failed to mark duplicate ids for nulling on ${field}`, () => {
    for (const row of rows) {
      const ids = parseAggregatedIds(row);
      const groupLabel = getDuplicateGroupLabel(row, field);
      const count = Number(row.count ?? row.dataValues?.count ?? 0);

      if (ids.length === 0) {
        logger.warn(
          `Skipping duplicate group for ${field}="${groupLabel}": no parseable crew ids (count=${count})`,
        );
        continue;
      }

      if (count > 1 && ids.length < 2) {
        logger.warn(
          `Duplicate group for ${field}="${groupLabel}" has count ${count} but only ${ids.length} parseable id(s)`,
        );
      }

      for (const id of ids) {
        setFieldNullOnCrew(duplicateUpdate, id, field);
      }
    }
  });
```

```
const buildDuplicateUpdateMap = (groups: DuplicateGroupsResult): DuplicateUpdateMap =>
  runSync('Failed to build duplicate crew update map', () => {
    const duplicateUpdate: DuplicateUpdateMap = {};

    for (const field of DUPLICATE_FIELDS) {
      if (groups[field].length > 0) {
        markDuplicateIdsForNulling(groups[field], field, duplicateUpdate);
      }
    }

    return duplicateUpdate;
  });

const updateCrewDuplicateRecord = (
  crewId: CrewListAttributes['id'],
  update: CrewDuplicateNullUpdate,
  transaction: Transaction,
): Promise<void> =>
  runAsync(`Failed to update crew duplicate fields for id ${crewId}`, async () => {
    const [affectedCount] = await CrewDetails.update(
      update as Parameters<typeof CrewDetails.update>[0],
      {
        where: { id: crewId },
        transaction,
      },
    );

    if (!affectedCount) {
      throw new Error(`No crew row updated for id ${crewId}`);
    }
  });

const rollbackTransaction = (transaction: Transaction): Promise<void> =>
  runAsync('Failed to roll back crew duplicate update transaction', () =>
    transaction.rollback(),
  );

const applyCrewDuplicateUpdates = (
  duplicateUpdate: DuplicateUpdateMap,
): Promise<void> =>
  runAsync('Failed to apply crew duplicate updates', async () => {
    const crewIds = Object.keys(duplicateUpdate) as CrewListAttributes['id'][];
    const transaction = await sequelize.transaction();

    try {
      logger.info(
        `Applying crew duplicate updates in transaction (${crewIds.length} crew)`,
      );

      for (let i = 0; i < crewIds.length; i += UPDATE_BATCH_SIZE) {
        const batch = crewIds.slice(i, i + UPDATE_BATCH_SIZE);
        await Promise.all(
          batch.map((crewId) =>
            updateCrewDuplicateRecord(
              crewId,
              duplicateUpdate[crewId],
              transaction,
            ),
          ),
        );
      }

      await transaction.commit();
      logger.info(`Committed crew duplicate updates for ${crewIds.length} crew`);
    } catch (error: unknown) {
      await rollbackTransaction(transaction).catch(() => undefined);
      logger.error(
```

```
        `Rolled back crew duplicate updates (${crewIds.length} crew); no partial changes applied:
${getErrorMessage(error)}`,
      );
      throw error;
    }
  });

/**
 * Runs once at server startup (after DB init) to clear duplicate unique values before constraints apply.
 * Sets `passport_no`, `seaman_book_no`, and/or `payroll_no` to null on every crew row in a duplicate group.
 * Returns `''` if there is nothing to update; `'Successfully Updated'` if all changes committed; throws on failure.
 */
export const uniqueConstraintDuplicateCrewUpdate = (): Promise<string> =>
  runAsync('Crew Unique Constraint Duplicate Update', async () => {
    logger.info('Crew Unique Constraint Duplicate Update Start');

    const groups = await findAllDuplicateGroups();

    if (!hasDuplicateGroups(groups)) {
      logger.info(
        `No Crew Duplicate To Update - Passport ${groups.passport_no.length}, ` +
        `Seaman ${groups.seaman_book_no.length}, Payroll ${groups.payroll_no.length}`,
      );
      return '';
    }

    const duplicateUpdate = buildDuplicateUpdateMap(groups);
    const crewIds = Object.keys(duplicateUpdate);

    if (crewIds.length === 0) {
      throw new Error(
        'Duplicate groups were found but no crew ids could be resolved from array_agg results',
      );
    }

    await applyCrewDuplicateUpdates(duplicateUpdate);

    logger.info(
      `Crew Unique Constraint Duplicate Update End - ${crewIds.length} crew updated`,
    );
    return 'Successfully Updated';
  });
```

Summary of enhancements

Area	Original	Enhanced
Logic guard	AND — all 3 fields must have duplicates	OR — any field with duplicates
Data safety	No transaction	Single DB transaction + rollback
Query filters	None	Excludes null/empty, deleted, uses LOWER(TRIM())
Types	`any`	`CrewListAttributes`-based types
Updates	Sequential, no affected-row check	Batched parallel (50), validates rows updated
Errors	Empty rethrow	Contextual logging per step
Structure	One 104-line function	~15 focused helpers

---

4. Productivity Observation

Using staged AI prompts turned a single brittle utility into a production-ready module in far less time than a manual rewrite would take: each prompt targeted one concern (bugs, types, transactions, performance, lint), which kept reviews small and reduced regression risk. The line count grew (~104 → ~321), but most of that is structure, types, and guards that would otherwise surface as production incidents or support time. **Net effect:** faster delivery of correct behavior, with the caveat that the job should still be validated on staging database.

Generated for PDF export. Open this file in Word, VS Code, or any Markdown-to-PDF tool (e.g. Print → Save as PDF).